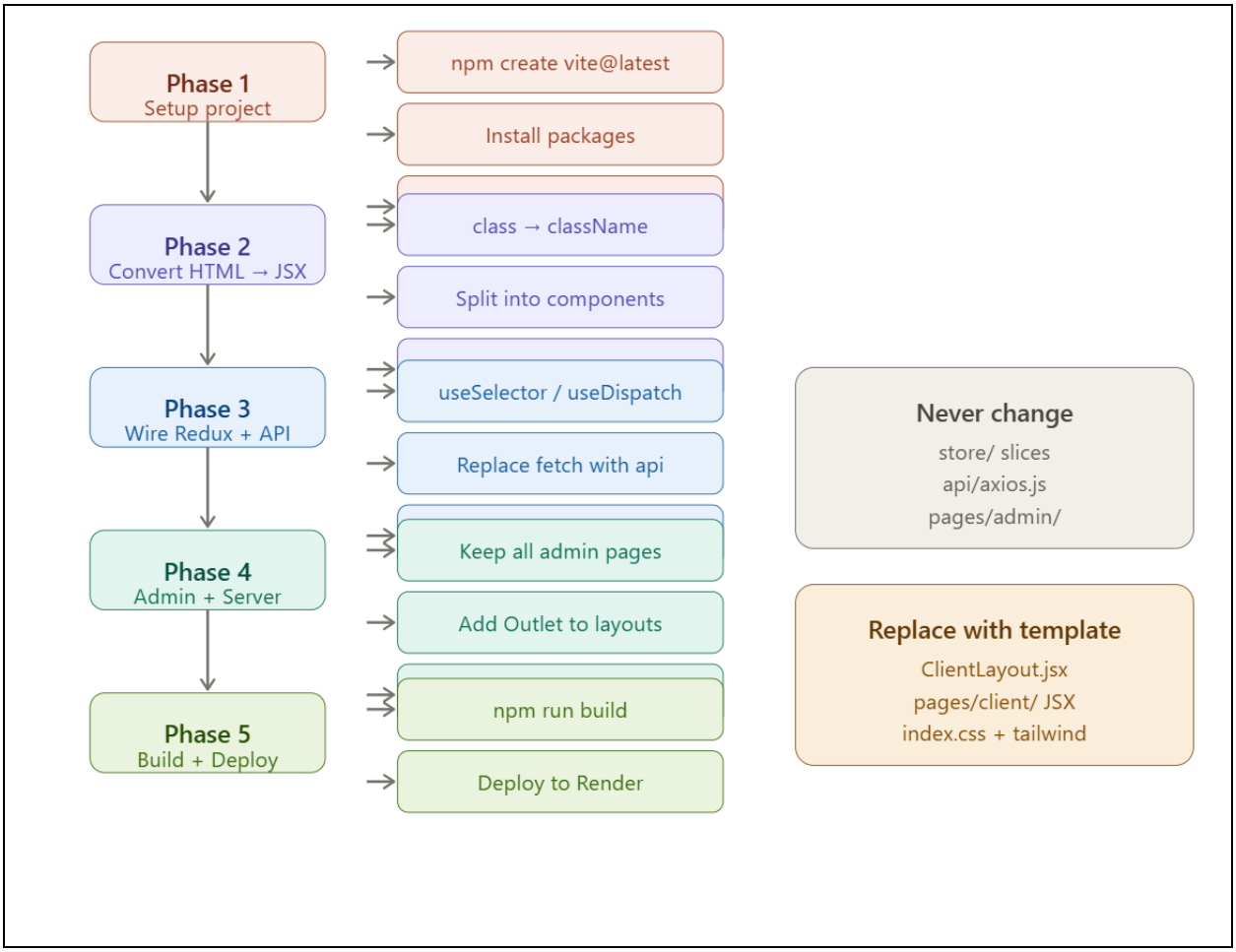
Convert an HTML/CSS template into a React Vite template and then integrate it with backend. How do I do this?



Now let me write the complete, visual step-by-step guide:Here is the complete step-by-step guide to convert your HTML/CSS template and integrate it:

Phase 1 — Project Setup

Step 1: Create a fresh Vite project

```
npm create vite@latest marketpro-client -- --template react
cd marketpro-client
```

Step 2: Install all required packages

```
# Core (required — do not skip any)
npm install @reduxjs/toolkit react-redux react-router-dom axios react-hot-
toast

# Add your template's dependencies
npm install swiper          # if template uses sliders
npm install framer-motion   # if template uses animations
npm install lucide-react    # for icons
```

Step 3: Copy the data layer from the existing project — never rewrite these

```
# Copy these folders exactly as-is into your new src/
cp -r old-project/src/store       src/store
cp -r old-project/src/api         src/api
cp -r old-project/src/pages/admin src/pages/admin
cp    old-project/src/App.jsx     src/App.jsx
cp    old-project/src/main.jsx    src/main.jsx
```

Step 4: Create `vite.config.js`

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'path'

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') }  // ← must keep
  },
  server: {
    proxy: { '/api': { target: 'http://localhost:8000', changeOrigin: true }
}
  }
})
```

Step 5: Create `.env.local`

```
VITE_API_URL=http://localhost:8000/api
```

Phase 2 — Convert HTML Template to JSX

This is where you do the actual template conversion. Go file by file.

Step 6: Identify your template's structure

Open your HTML template and map every section to a file:

```
HTML template structure          →   React file to create
──────────────────────────────────────────────────────────
<header> ... </header>           →   part of ClientLayout.jsx
<footer> ... </footer>           →   part of ClientLayout.jsx
index.html (homepage content)    →   pages/client/HomePage.jsx
products.html                    →   pages/client/ProductsPage.jsx
product-detail.html              →   pages/client/ProductDetail.jsx
cart.html                        →   pages/client/CartPage.jsx
wishlist.html                    →   pages/client/WishlistPage.jsx
checkout.html                    →   pages/client/CheckoutPage.jsx
orders.html                      →   pages/client/OrdersPage.jsx
profile.html                     →   pages/client/ProfilePage.jsx
login.html                       →   pages/client/LoginPage.jsx
register.html                    →   pages/client/RegisterPage.jsx

Repeated card/product snippet    →   components/product/ProductCard.jsx
Cart slide-in panel              →   components/cart/CartDrawer.jsx
```

Step 7: The 8 HTML → JSX conversion rules

Take each HTML section and make these replacements:

```
HTML                             →    JSX
──────────────────────────────────────────────────────────
class="..."                      →    className="..."
for="..."  (label)               →    htmlFor="..."
<img src="...">                  →    <img src="..." alt="..." />  (self-close)
<input>  <br>  <hr>              →    <input />  <br />  <hr />
onclick="fn()"                   →    onClick={() => fn()}
onchange="fn()"                  →    onChange={(e) => fn(e)}
style="color:red"                →    style={{ color: 'red' }}
<!-- comment -->                 →    {/* comment */}
```

Step 8: Convert `<head>` content

Move everything from `<head>` into `index.html` in your Vite project:

```
<!-- index.html (Vite entry) -->
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>MarketPro</title>

    <!-- Paste your template's Google Fonts link here -->
```

```
    <link href="https://fonts.googleapis.com/css2?family=..."
rel="stylesheet" />

    <!-- Any template CDN CSS (Bootstrap, etc.) -->
    <link rel="stylesheet" href="https://..." />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Step 9: Move CSS into `src/index.css`

Copy all your template's CSS into `src/index.css`. If using Tailwind keep the directives at top:

```
/* src/index.css */
@tailwind base;
@tailwind components;
@tailwind utilities;

/* ↓ Paste all your template's custom CSS below ↓ */
:root {
  --primary: #f97316;
  --font-main: 'Your Template Font', sans-serif;
}

/* your template's .navbar, .card, .btn styles etc. */
```

If your template uses a separate CSS file instead of Tailwind, import it in `main.jsx`:

```
// src/main.jsx
import './index.css'
import './your-template.css'   // ← add this line
```

Phase 3 — Wire Redux + API into Each Component

Step 10: Build `ClientLayout.jsx` — the shell that wraps all pages

Take your template's `<header>` and `<footer>` HTML, convert to JSX, then add these 5 wires:

```
// src/components/layout/ClientLayout.jsx
import { Outlet } from 'react-router-dom'          // WIRE 1 — renders pages
import { Link, useNavigate } from 'react-router-dom'
import { useSelector, useDispatch } from 'react-redux'
import { logout } from '@/store/slices/authSlice'
import { toggleCart } from '@/store/slices/uiSlice'
import CartDrawer from '@/components/cart/CartDrawer' // WIRE 2 — cart drawer

export default function ClientLayout() {
  const dispatch = useDispatch()
```

```
  const { user } = useSelector(s => s.auth)              // WIRE 3 — user
  const { item_count } = useSelector(s => s.cart)        // WIRE 4 — cart
badge
  const { ids: wishIds } = useSelector(s => s.wishlist)  // WIRE 5 — wishlist
badge

  return (
    <>
      {/* ── Paste your template's header JSX here ── */}
      <header className="your-template-header-class">
        <Link to="/">Logo</Link>

        {/* Cart button → dispatch(toggleCart()) */}
        <button onClick={() => dispatch(toggleCart())}>
          Cart <span>{item_count}</span>
        </button>

        {/* Wishlist */}
        <Link to="/wishlist">
          Wishlist <span>{wishIds.length}</span>
        </Link>

        {/* User menu */}
        {user ? (
          <>
            <Link to="/profile">{user.name}</Link>
            <Link to="/orders">Orders</Link>
            {(user.role === 'admin') && <Link to="/admin">Admin</Link>}
            <button onClick={() => dispatch(logout())}>Logout</button>
          </>
        ) : (
          <Link to="/login">Sign In</Link>
        )}
      </header>

      <CartDrawer />  {/* ← must be here, hidden until cart opens */}

      <main>
        <Outlet />    {/* ← renders current page here */}
      </main>

      {/* ── Paste your template's footer JSX here ── */}
      <footer className="your-template-footer-class">
        Footer content...
      </footer>
    </>
  )
}
```

Step 11: Convert each page — replace static HTML with live data

For every client page, the pattern is the same:

```
1. Take the page's HTML from your template
2. Wrap it in a React function component
3. Add the data hook at the top
```

```
4. Replace hardcoded text with {variables}
5. Replace hardcoded links with <Link to="...">
6. Replace onclick with onClick dispatching the right action
```

Here is every page with its exact hook:

HomePage.jsx

```
import { useEffect } from 'react'
import { useDispatch, useSelector } from 'react-redux'
import { fetchProducts } from '@/store/slices/productSlice'

export default function HomePage() {
  const dispatch = useDispatch()
  const { list: products, categories, loading } = useSelector(s =>
s.products)

  useEffect(() => {
    dispatch(fetchProducts({ featured: true, limit: 8 }))
  }, [dispatch])

  return (
    <>
      {/* Your template's homepage HTML converted to JSX */}
      {/* Replace hardcoded product cards with: */}
      {loading
        ? <p>Loading...</p>
        : products.map(p => <ProductCard key={p.id} product={p} />)
      }
      {/* Replace hardcoded category grid with: */}
      {categories.map(cat => (
        <Link key={cat.id} to={`/products?category=${cat.id}`}>
          {cat.name}
        </Link>
      ))}
    </>
  )
}
```

ProductsPage.jsx

```
import { useEffect } from 'react'
import { useDispatch, useSelector } from 'react-redux'
import { useSearchParams } from 'react-router-dom'
import { fetchProducts } from '@/store/slices/productSlice'

export default function ProductsPage() {
  const dispatch = useDispatch()
  const [searchParams, setSearchParams] = useSearchParams()
  const { list, pagination, categories, brands, loading } = useSelector(s =>
s.products)

  useEffect(() => {
    dispatch(fetchProducts(Object.fromEntries(searchParams.entries())))
  }, [searchParams, dispatch])
```

```
  const filter = (key, val) => {
    const p = new URLSearchParams(searchParams)
    val ? p.set(key, val) : p.delete(key)
    p.set('page', '1')
    setSearchParams(p)
  }

  return (
    <>
      {/* Your template's filter sidebar — wire each filter input: */}
      <select onChange={e => filter('category', e.target.value)}>
        <option value="">All Categories</option>
        {categories.map(c => <option key={c.id}
value={c.id}>{c.name}</option>)}
      </select>

      {/* Sort dropdown */}
      <select onChange={e => filter('sort', e.target.value)}>
        <option value="createdAt">Latest</option>
        <option value="price_asc">Price: Low to High</option>
        <option value="price_desc">Price: High to Low</option>
      </select>

      {/* Product grid */}
      {list.map(p => <ProductCard key={p.id} product={p} />)}

      {/* Pagination */}
      {Array.from({ length: pagination.pages || 1 }, (_, i) => (
        <button key={i} onClick={() => filter('page', i + 1)}>{i +
1}</button>
      ))}
    </>
  )
}
```

ProductDetail.jsx

```
import { useEffect, useState } from 'react'
import { useParams } from 'react-router-dom'
import { useDispatch, useSelector } from 'react-redux'
import { fetchProduct, clearCurrentProduct } from
'@/store/slices/productSlice'
import { addToCart } from '@/store/slices/cartSlice'
import { toggleWishlist } from '@/store/slices/wishlistSlice'

export default function ProductDetail() {
  const { id } = useParams()
  const dispatch = useDispatch()
  const [qty, setQty] = useState(1)
  const { currentProduct: p, productLoading } = useSelector(s => s.products)
  const { ids: wishIds } = useSelector(s => s.wishlist)

  useEffect(() => {
    dispatch(fetchProduct(id))
    return () => dispatch(clearCurrentProduct())
```

```
  }, [id, dispatch])

  if (productLoading) return <p>Loading...</p>
  if (!p) return <p>Not found</p>

  return (
    <>
      {/* Template's product gallery — use p.images[] array */}
      {p.images.map((url, i) => <img key={i} src={url} alt={p.name} />)}

      {/* YouTube video embed — only show if p.youtube_url exists */}
      {p.youtube_url && (
        <iframe
          src={p.youtube_url.replace('watch?v=', 'embed/')}
          allowFullScreen
        />
      )}

      <h1>{p.name}</h1>
      <p>{p.price}</p>

      {/* Variants */}
      {p.variants?.map(v => (
        <button key={v.name}>{v.name} — ₹{v.price}</button>
      ))}

      {/* Add to cart */}
      <input type="number" value={qty} onChange={e =>
setQty(Number(e.target.value))} min="1" />
      <button onClick={() => dispatch(addToCart({ product_id: p.id, quantity:
qty }))}>
        Add to Cart
      </button>

      {/* Wishlist */}
      <button onClick={() => dispatch(toggleWishlist(p.id))}>
        {wishIds.includes(p.id) ? '♥ Remove' : '♡ Wishlist'}
      </button>
    </>
  )
}
```

LoginPage.jsx

```
import { useState } from 'react'
import { useDispatch, useSelector } from 'react-redux'
import { useNavigate } from 'react-router-dom'
import { login } from '@/store/slices/authSlice'

export default function LoginPage() {
  const dispatch = useDispatch()
  const navigate = useNavigate()
  const { loading } = useSelector(s => s.auth)
  const [email, setEmail] = useState('')
  const [password, setPassword] = useState('')
```

```
  const handleSubmit = async (e) => {
    e.preventDefault()
    const result = await dispatch(login({ email, password }))
    if (!result.error) navigate('/')
  }

  return (
    <form onSubmit={handleSubmit}>
      {/* Your template's login form HTML → JSX */}
      <input type="email" value={email} onChange={e =>
setEmail(e.target.value)} />
      <input type="password" value={password} onChange={e =>
setPassword(e.target.value)} />
      <button type="submit" disabled={loading}>
        {loading ? 'Signing in...' : 'Sign In'}
      </button>
    </form>
  )
}
```

CartPage.jsx

```
import { useSelector, useDispatch } from 'react-redux'
import { updateCartItem, clearCart } from '@/store/slices/cartSlice'
import { Link } from 'react-router-dom'

export default function CartPage() {
  const dispatch = useDispatch()
  const { items, total } = useSelector(s => s.cart)

  return (
    <>
      {items.map(item => (
        <div key={item.product_id}>
          <img src={item.product?.images?.[0]} alt={item.product?.name} />
          <span>{item.product?.name}</span>
          {/* Decrease — if qty becomes 0, item is removed */}
          <button onClick={() =>
            dispatch(updateCartItem({ productId: item.product_id, quantity:
item.quantity - 1 }))
          }>-</button>
          <span>{item.quantity}</span>
          <button onClick={() =>
            dispatch(updateCartItem({ productId: item.product_id, quantity:
item.quantity + 1 }))
          }>+</button>
          <span>₹{item.product?.price * item.quantity}</span>
        </div>
      ))}
      <div>Total: ₹{total}</div>
      <Link to="/checkout">Proceed to Checkout</Link>
    </>
  )
}
```

WishlistPage.jsx

```
import { useSelector, useDispatch } from 'react-redux'
import { toggleWishlist } from '@/store/slices/wishlistSlice'
import { addToCart } from '@/store/slices/cartSlice'
import { Link } from 'react-router-dom'

export default function WishlistPage() {
  const dispatch = useDispatch()
  const { items } = useSelector(s => s.wishlist)

  return (
    <>
      {items.map(item => (
        <div key={item.product_id}>
          <Link to={`/products/${item.product_id}`}>
            <img src={item.product?.images?.[0]} alt={item.product?.name} />
            <span>{item.product?.name}</span>
            <span>₹{item.product?.price}</span>
          </Link>
          <button onClick={() => dispatch(addToCart({ product_id:
item.product_id, quantity: 1 }))}>
            Add to Cart
          </button>
          <button onClick={() => dispatch(toggleWishlist(item.product_id))}>
            Remove
          </button>
        </div>
      ))}
    </>
  )
}
```

CheckoutPage.jsx

```
import { useState } from 'react'
import { useSelector, useDispatch } from 'react-redux'
import { useNavigate } from 'react-router-dom'
import { fetchCart } from '@/store/slices/cartSlice'
import api from '@/api/axios'
import toast from 'react-hot-toast'

export default function CheckoutPage() {
  const dispatch = useDispatch()
  const navigate = useNavigate()
  const { items, total } = useSelector(s => s.cart)
  const [address, setAddress] = useState({ street:'', city:'', state:'',
country:'India', zip_code:'' })
  const [payment, setPayment] = useState('cod')
  const [loading, setLoading] = useState(false)

  const placeOrder = async (e) => {
    e.preventDefault()
    setLoading(true)
    try {
      const res = await api.post('/orders', {
        shipping_address: { ...address, label: 'Home', is_default: false },
        payment_method: payment
```

```
      })
      dispatch(fetchCart())                          // clears cart
      navigate(`/orders/${res.data.data.id}`)
      toast.success('Order placed!')
    } catch (err) {
      toast.error(err.response?.data?.message || 'Failed')
    }
    setLoading(false)
  }

  return (
    <form onSubmit={placeOrder}>
      {/* Template's address form fields — bind each to address state */}
      <input value={address.street} onChange={e => setAddress(a => ({...a,
street: e.target.value}))} />
      <input value={address.city}   onChange={e => setAddress(a => ({...a,
city: e.target.value}))}   />
      <input value={address.state}  onChange={e => setAddress(a => ({...a,
state: e.target.value}))}  />
      <input value={address.zip_code} onChange={e => setAddress(a => ({...a,
zip_code: e.target.value}))} />

      {/* Payment method radio buttons */}
      {['cod','upi','card'].map(m => (
        <label key={m}>
          <input type="radio" value={m} checked={payment === m} onChange={()
=> setPayment(m)} />
          {m.toUpperCase()}
        </label>
      ))}

      <div>Total: ₹{total}</div>
      <button type="submit" disabled={loading}>{loading ? 'Placing...' :
'Place Order'}</button>
    </form>
  )
}
```

OrdersPage.jsx

```
import { useEffect, useState } from 'react'
import { Link } from 'react-router-dom'
import api from '@/api/axios'

export default function OrdersPage() {
  const [orders, setOrders] = useState([])
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    api.get('/orders').then(r => setOrders(r.data.data)).finally(() =>
setLoading(false))
  }, [])

  return (
    <>
      {orders.map(order => (
```

```
        <Link key={order.id} to={`/orders/${order.id}`}>
          #{order.id.slice(-6)}  {order.status}  ₹{order.total}
        </Link>
      ))}
    </>
  )
}
```

Step 12: Build `ProductCard.jsx`

Take the single product card HTML from your template and convert it:

```
import { Link } from 'react-router-dom'
import { useDispatch, useSelector } from 'react-redux'
import { addToCart } from '@/store/slices/cartSlice'
import { toggleWishlist } from '@/store/slices/wishlistSlice'

export default function ProductCard({ product }) {
  const dispatch = useDispatch()
  const { ids: wishIds } = useSelector(s => s.wishlist)
  const isWished = wishIds.includes(product.id)

  const discount = product.compare_price > product.price
    ? Math.round(((product.compare_price - product.price) /
product.compare_price) * 100)
    : 0

  return (
    // Your template's card HTML → JSX
    // Replace static values with these variables:
    //   product.images?.[0]           → main image URL
    //   product.name                  → product title
    //   product.price                 → sale price
    //   product.compare_price         → original price (show only if >
price)
    //   discount                      → "20% off" badge
    //   product.is_new_arrival        → "NEW" badge
    //   product.is_on_sale            → "SALE" badge
    //   product.stock === 0           → show "Out of Stock"
    //   product.avg_rating            → star rating (0-5)
    //   product.brand?.name           → brand name above title
    //   isWished                      → filled vs outline heart icon
    //
    // Replace static hrefs/onclick with:
    //   <Link to={`/products/${product.id}`}>  → card click
    //   onClick={() => dispatch(addToCart({product_id: product.id, quantity:
1}))
    //   onClick={() => dispatch(toggleWishlist(product.id))}
    <div className="your-card-class">
      <Link to={`/products/${product.id}`}>
        <img src={product.images?.[0]} alt={product.name} />
      </Link>
      {discount > 0 && <span>{discount}% OFF</span>}
      <h3>{product.name}</h3>
      <span>₹{product.price}</span>
```

```
      {product.compare_price > product.price &&
<del>₹{product.compare_price}</del>}
      <button onClick={() => dispatch(addToCart({ product_id: product.id,
quantity: 1 }))}>
        Add to Cart
      </button>
      <button onClick={() => dispatch(toggleWishlist(product.id))}>
        {isWished ? '♥' : '♡'}
      </button>
    </div>
  )
}
```

Step 13: Build `CartDrawer.jsx`

Take your template's slide-in cart panel HTML, convert to JSX, add these:

```
import { useSelector, useDispatch } from 'react-redux'
import { closeCart } from '@/store/slices/uiSlice'
import { updateCartItem } from '@/store/slices/cartSlice'
import { Link } from 'react-router-dom'

export default function CartDrawer() {
  const dispatch = useDispatch()
  const { cartOpen } = useSelector(s => s.ui)       // controls open/close
  const { items, total } = useSelector(s => s.cart) // live cart data

  return (
    // Your template's cart panel HTML → JSX
    // Show/hide using cartOpen:
    <div className={`cart-panel ${cartOpen ? 'open' : ''}`}>
      <button onClick={() => dispatch(closeCart())}>✕</button>

      {items.map(item => (
        <div key={item.product_id}>
          <img src={item.product?.images?.[0]} alt={item.product?.name} />
          <span>{item.product?.name}</span>
          <button onClick={() => dispatch(updateCartItem({
            productId: item.product_id, quantity: item.quantity - 1
          }))}>-</button>
          <span>{item.quantity}</span>
          <button onClick={() => dispatch(updateCartItem({
            productId: item.product_id, quantity: item.quantity + 1
          }))}>+</button>
        </div>
      ))}

      <div>Total: ₹{total}</div>
      <Link to="/checkout" onClick={() => dispatch(closeCart())}>
        Checkout
      </Link>
    </div>
  )
}
```

Phase 4 — Admin & Server Stay Untouched

Step 14: Keep admin pages as-is

All files in `src/pages/admin/` were already copied in Step 3. Do not modify them. Just make sure `AdminLayout.jsx` still has `<Outlet />`:

```
// src/components/layout/AdminLayout.jsx
// You can restyle this sidebar with your template's admin design
// But these 3 things must stay:

import { Outlet } from 'react-router-dom'    // renders admin page content
import { useDispatch, useSelector } from 'react-redux'
import { logout } from '@/store/slices/authSlice'

// Keep all 7 nav links pointing to these exact paths:
const NAV = [
  { label: 'Dashboard',      to: '/admin' },
  { label: 'Products',       to: '/admin/products' },
  { label: 'Categories',     to: '/admin/categories' },
  { label: 'Brands',         to: '/admin/brands' },
  { label: 'Orders',         to: '/admin/orders' },
  { label: 'Customers',      to: '/admin/customers' },
  { label: 'Wishlist Stats', to: '/admin/wishlist-stats' },
]

export default function AdminLayout() {
  const dispatch = useDispatch()
  const { user } = useSelector(s => s.auth)

  return (
    <div style={{ display: 'flex' }}>
      <aside>
        {/* Your new admin sidebar design here */}
        {NAV.map(n => <Link key={n.to} to={n.to}>{n.label}</Link>)}
        <button onClick={() => dispatch(logout())}>Logout</button>
      </aside>
      <main style={{ flex: 1 }}>
        <Outlet />   {/* ← required */}
      </main>
    </div>
  )
}
```

Phase 5 — Build & Deploy

Step 15: Test locally

```
# Terminal 1 — start the Python server
cd server
uvicorn main:app --reload
```

```
# Terminal 2 — start the React client
cd client
npm run dev
```

Visit `http://localhost:5173` for the store and `http://localhost:5173/admin` for the dashboard.

Step 16: Build for production

```
cd client
npm run build      # outputs to dist/ folder
```

Step 17: Deploy to Render

Create a `render.yaml` in the client folder:

```
services:
  - type: web
    name: marketpro-client
    runtime: static
    buildCommand: npm install && npm run build
    staticPublishPath: dist
    envVars:
      - key: VITE_API_URL
        value: https://your-server.onrender.com/api
    routes:
      - type: rewrite
        source: /*
        destination: /index.html
```

Quick Reference — What Each File Does

File	Your job
`store/` and `api/axios.js`	Copy as-is, never edit
`App.jsx`	Copy as-is, only change import paths if you rename files
`main.jsx`	Copy as-is, only add your CSS import
`index.html`	Paste your template's font links and CDN CSS here
`index.css`	Paste your template's CSS here
`ClientLayout.jsx`	Replace with template header/footer + add 5 wires
`AdminLayout.jsx`	Replace sidebar design + keep `<Outlet />` and 7 nav links
`CartDrawer.jsx`	Replace design + keep `cartOpen`, `items`, `updateCartItem`
`ProductCard.jsx`	Replace card design + keep `addToCart`, `toggleWishlist`
`pages/client/*.jsx`	Replace with template page HTML + add data hook per page
`pages/admin/*.jsx`	Copy as-is, never edit